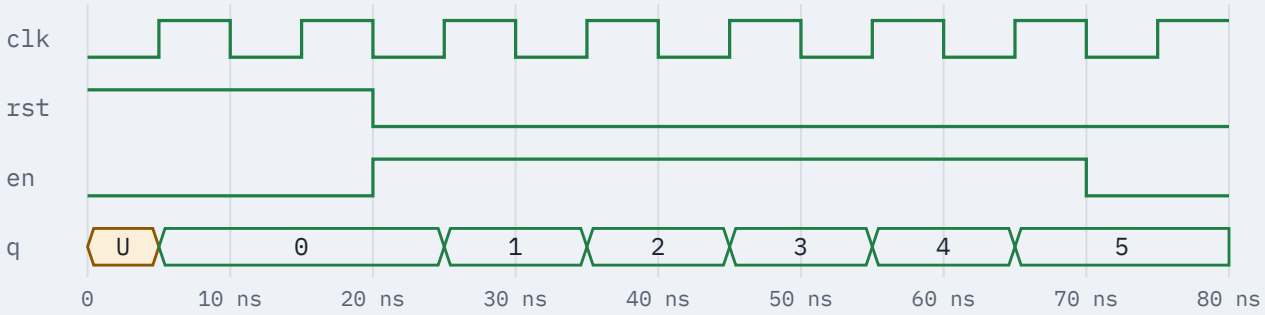
KIT PORTÁTIL · NOTEPAD++ + GHDL + GTKWAVE

Manual del kit VHDL

Un entorno ligero para escribir, simular y ver las formas de onda de tus diseños VHDL sin instalar Vivado: funciona desde el pendrive o desde una copia en tu ordenador.



Los primeros 80 ns del ejemplo del contador, como los verás en GTKWave: la cuenta vale U (sin valor) hasta el primer flanco con reset, y sube de 0 a 5 mientras `en` vale 1.

1 Qué hay en el pendrive

Todo está dentro de la carpeta `VHDL` del pendrive. La letra de la unidad cambia según el PC (`D:`, `E:`, `F:`…); da igual, el kit funciona con cualquiera.

ELEMENTO	PARA QUÉ SIRVE
`abrir_editor.bat`	Abre el editor Notepad++ con tus proyectos. **Empieza siempre por aquí.**
`proyectos\`	Tus proyectos, una carpeta por proyecto. Trae cuatro: `ejemplo_mux`, `ejemplo_contador`, `reloj_pulsera` (un ejercicio de máquina de estados, con su enunciado en `ENUNCIADO.txt`) y `sobel_bordes` (detección de bordes en una imagen; los pasos están en su `LEEME.txt`).
`sim.bat`	Compila, simula y abre las ondas. Lo usa la tecla `F6` del editor.

ELEMENTO	PARA QUÉ SIRVE
`comprobar.bat`	Comprueba un fichero `.vhd` sin simular. Lo usa la tecla `F9`.
`limpiar.bat`	Borra los ficheros que genera la simulación.
`LEEME.txt`	Resumen rápido de este manual.
`npp\`, `ghdl\`, `gtkwave\`, `python\`	Los programas: el editor, el simulador, el visor de ondas y Python (lo usan los scripts de algunos proyectos, como `sobel_bordes`). No hace falta tocarlos.

PASO 1 · NOTEPAD++

Escribe

El diseño (`xxx.vhd`) y su banco de pruebas (`tb_xxx.vhd`).

PASO 2 · GHDL

Pulsa F6

Guarda, compila y simula. Los mensajes salen en la consola del editor.

PASO 3 · GTKWAVE

Mira las ondas

Se abre solo, con la simulación completa en pantalla.

2 Tu primera simulación

Hazlo con el ejemplo del multiplexor para comprobar que todo funciona en tu PC.

1. Conecta el pendrive, abre la carpeta `VHDL` y haz doble clic en `abrir_editor.bat`.

   Se abre Notepad++ con la carpeta `proyectos` en el panel de la izquierda.

2. En ese panel, despliega `ejemplo_mux` y haz doble clic en `tb_mux.vhd`. Es el **banco de pruebas** (testbench): el fichero que genera los estímulos y comprueba el diseño `mux4.vhd`.

3. Con la pestaña `tb_mux.vhd` activa, pulsa `F6`.

   En la parte inferior aparece la consola con el progreso. Si todo va bien, termina así:

```
=== 1/3  Importando los ficheros .vhd ===
  mux4.vhd
  tb_mux.vhd
=== 2/3  Compilando tb_mux ===
=== 3/3  Simulando tb_mux hasta 1us ===

Abriendo GTKWave con la vista guardada tb_mux.gtkw

Simulacion terminada correctamente.
<<< Process finished (PID=48096). (Exit code 0)
================ READY ================
```

4. Se abre GTKWave, maximizado, con el selector, las cuatro entradas y la salida. Comprueba que `y` copia la entrada que indica `sel`.

Si has llegado hasta aquí, el kit funciona en este PC.

Repite con `ejemplo_contador\tb_contador.vhd` para ver un circuito con reloj.

F6 simula 1 µs, salvo que el proyecto indique otro tiempo

Es suficiente para casi todos los ejemplos. Si tu banco de pruebas necesita más tiempo, la simulación se corta en 1 µs y verás `simulation stopped by --stop-time @1us`. Para simular más con F6, crea en la carpeta del proyecto un fichero `tiempo_sim.txt` con el tiempo en la primera línea (por ejemplo `200us`), como en `sobel_bordes`. También puedes usar la consola (sección 7) e indicar el tiempo, por ejemplo `5us`.

3 Comprobar un fichero sin simular

Con `F9` compruebas el fichero activo **sin banco de pruebas**: GHDL revisa la sintaxis, los nombres no declarados, los tipos y los nombres de los puertos, pero no simula nada. Úsalo a menudo mientras escribes un diseño.

1. Abre el fichero que quieres revisar: el diseño (`mux4.vhd`) o el banco de pruebas.
2. Pulsa `F9` (o `Plugins > NppExec > Comprobar`). Se guardan los ficheros y el resultado sale en la consola:

```
=== Comprobando mux4.vhd ===
Correcto: mux4.vhd no tiene errores de sintaxis, nombres, tipos ni
puertos.
```

Si hay un error, GHDL indica el fichero, la línea y la columna:

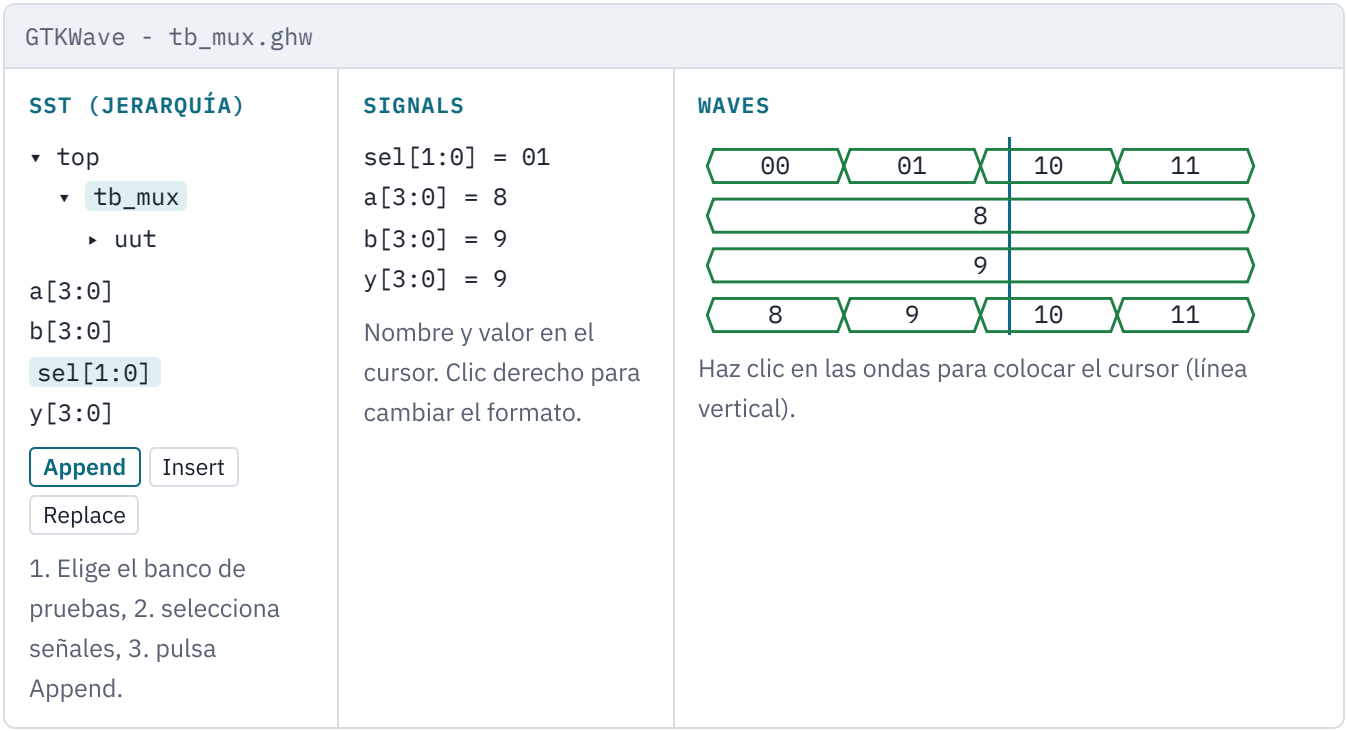
```
=== Comprobando contador.vhd ===
contador.vhd:36:29:error: missing ";" at end of statement
        cuenta <= cuenta + 1         -- despues de 15 vuelve a 0
                            ^
```

> **Si el fichero usa otras entidades**
>
> Un banco de pruebas usa el diseño que prueba. Los ficheros con esas entidades tienen que estar en la misma carpeta y no tener errores; si no, verás `unit "contador" not found in library "work"`. Comprueba primero el diseño y después el banco de pruebas.

Que un fichero sea correcto no significa que el circuito haga lo que quieres: para ver su comportamiento, simula con `F6`. Desde la consola: `..\..\comprobar.bat mux4.vhd`

4 Ver las ondas en GTKWave

GTKWave abre la simulación completa en pantalla. Si el proyecto tiene una vista guardada (`tb_xxx.gtkw`), aparecen ya sus señales; si no, la zona de ondas sale vacía y tienes que añadirlas.



Añadir señales

1. En el panel **SST** (arriba a la izquierda), despliega `top` y haz clic en tu banco de pruebas, por ejemplo `tb_mux`. Debajo aparecen sus señales.
2. Selecciona las que quieras (`Ctrl`+clic para varias) y pulsa **Append**. También puedes arrastrarlas a la zona de ondas.
3. Las señales internas del diseño están dentro de `uut`, el nombre de la instancia en el banco de pruebas.

Moverse por las ondas

- **Ver toda la simulación:** menú `Time > Zoom > Zoom Best Fit` (`Shift`+`Alt`+`F`).
- **Acercar y alejar:** `Ctrl`+`+` y `Ctrl`+`-`, o los botones de lupa de la barra.
- **Leer un valor:** haz clic en un instante de la zona de ondas. La columna Signals muestra el valor de cada señal en ese instante.
- **Cambiar el formato de un bus:** clic derecho sobre la señal > `Data Format` > `Binary`, `Decimal` o `Hex`.

Guardar la vista para la próxima vez

Menú `File > Write Save File As…` y guárdala en la carpeta del proyecto con el nombre del banco de pruebas: `tb_xxx.gtkw`. A partir de entonces, cada simulación abre GTKWave

con esas señales y formatos. Después, `File > Write Save File` (`Ctrl`+`S`) actualiza la vista.

Volver a simular con GTKWave abierto

Cada `F6` abre una ventana nueva de GTKWave. Si prefieres seguir en la que tienes, tras simular usa `File > Reload Waveform` (`Ctrl`+`Shift`+`R`): actualiza las ondas sin perder las señales que añadiste. Cierra las ventanas que sobren.

5 Crear tu propio proyecto

1. Crea una carpeta nueva dentro de `proyectos`. Usa un nombre corto, sin tildes ni espacios: `practica1`, `sumador`…
2. Crea dentro dos ficheros: el diseño (`puerta_and.vhd`) y su banco de pruebas (`tb_puerta_and.vhd`). En Notepad++: `File > New` y después `File > Save As…` con extensión `.vhd`.
3. Con la pestaña del banco de pruebas activa, pulsa `F6`.

> **Regla obligatoria: el fichero se llama igual que su entidad**
>
> `tb_puerta_and.vhd` debe contener `entity tb_puerta_and is`. `F6` usa el nombre del fichero activo para saber qué entidad simular. Por convenio, el nombre de un banco de pruebas empieza por `tb_` (`tb_mux`) o termina en `_tb` (`reloj_pulsera_v1_tb`).

> **Un proyecto por carpeta**
>
> Se compilan **todos** los `.vhd` de la carpeta. No dejes copias antiguas con extensión `.vhd` (por ejemplo `mux4_viejo.vhd`): renómbralas a `.txt` o muévelas a otra carpeta.

Plantilla para empezar

Una puerta AND y su banco de pruebas, que prueba las cuatro combinaciones de entrada. Cópialos, cambia los nombres y adáptalos a tu diseño. Después, comprueba en GTKWave que `y` solo vale 1 cuando `a` y `b` valen 1.

puerta_and.vhd

```
library ieee;
use ieee.std_logic_1164.all;

entity puerta_and is
  port (
    a, b : in  std_logic;
    y    : out std_logic
  );
end entity puerta_and;

architecture rtl of puerta_and is
begin
  y <= a and b;
end architecture rtl;
```

tb_puerta_and.vhd

```
library ieee;
use ieee.std_logic_1164.all;

-- El banco de pruebas no tiene puertos
entity tb_puerta_and is
end entity tb_puerta_and;

architecture sim of tb_puerta_and is
  signal a, b : std_logic := '0';
  signal y    : std_logic;
begin

  -- Unidad bajo prueba (UUT)
  uut : entity work.puerta_and
    port map (a => a, b => b, y => y);

  -- Las cuatro combinaciones de entrada, 10 ns cada una
  estimulos : process
  begin
    a <= '0'; b <= '0';
    wait for 10 ns;
    a <= '0'; b <= '1';
    wait for 10 ns;
    a <= '1'; b <= '0';
    wait for 10 ns;
    a <= '1'; b <= '1';
    wait for 10 ns;
    wait;                       -- detiene el proceso
  end process estimulos;

end architecture sim;
```

Si el diseño tiene reloj, genera el reloj con `clk <= not clk after 5 ns;` y termina la prueba con `std.env.stop;`, como en `ejemplo_contador\tb_contador.vhd`. Sin eso, el reloj nunca se detiene y la simulación dura hasta el tiempo máximo.

Escribe los comentarios sin tildes ni eñes (`-- senal de reset`): así el código se ve igual en cualquier editor y en Vivado.

6 Mensajes y errores frecuentes

Los errores de GHDL tienen siempre la forma `fichero.vhd:línea:columna: mensaje`. Ve a esa línea en el editor (`Ctrl` + `G`) y revisa también la línea anterior: muchas veces el fallo está justo antes.

MENSAJE	QUÉ SIGNIFICA Y QUÉ HACER
error `mux4.vhd:30:9:error: ';' expected at end of signal assignment`	Error de sintaxis: falta un `;`. Está al final de la instrucción anterior a la línea 30.
error `cannot find entity or configuration tb_prueba`	No hay ninguna entidad con el nombre del fichero activo. Revisa la regla: `tb_prueba.vhd` debe contener `entity tb_prueba`.
error `unit "contador" not found in library "work"`	El banco de pruebas usa un diseño que no está en la carpeta, o cuya entidad tiene otro nombre. Copia el fichero del diseño a la carpeta del proyecto o corrige el nombre en `entity work.xxx`.
aviso `AVISO: "mux4" no parece un banco de pruebas`	Has pulsado `F6` con el diseño activo en vez del banco de pruebas. Activa la pestaña del banco de pruebas y vuelve a pulsar `F6`.
aviso `no se puede reemplazar tb_xxx.ghw`	Otro programa tiene abierto el fichero de ondas. Ciérralo y vuelve a simular.
normal `simulation stopped @230ns`	El banco de pruebas ha terminado con `std.env.stop`. Correcto.
normal `simulation stopped by --stop-time @1us`	Se ha llegado al tiempo máximo. Si la prueba no había terminado, indica más tiempo en `tiempo_sim.txt` o desde la consola.

7 Simular desde la consola

Es la alternativa a `F6`, útil para elegir el tiempo de simulación.

1. Abre la carpeta del proyecto en el Explorador de archivos, escribe `cmd` en la barra de direcciones y pulsa `Intro`. Se abre una consola en esa carpeta.
2. Escribe el nombre de la entidad del banco de pruebas y, si quieres, el tiempo:

   ```
   ..\..\sim.bat tb_contador 5us
   ```

   `..\..\` sube de la carpeta del proyecto a la carpeta `VHDL`, donde está `sim.bat`. Unidades de tiempo: `ns`, `us`, `ms`, sin espacio. Si no lo indicas, se usa el de `tiempo_sim.txt` (si el proyecto lo tiene) o `1us`.

Si hay errores, la consola espera a que pulses una tecla. GTKWave queda ligado a esa consola: ciérrala cuando termines con GTKWave, no antes.

8 Limpiar un proyecto

La simulación crea la carpeta `work\` y el fichero de ondas `tb_xxx.ghw`. Para borrarlos (por ejemplo antes de entregar la práctica o de copiar el proyecto):

- En Notepad++, con cualquier fichero del proyecto activo: `Plugins > NppExec > Limpiar`.
- O desde la consola en la carpeta del proyecto: `..\..\limpiar.bat`

Nunca borra tus `.vhd` ni tus vistas `.gtkw`. No hace falta limpiar para volver a simular: cada simulación empieza de cero.

9 Si algo no funciona

PROBLEMA	SOLUCIÓN
`F6` o `F9` no hacen nada, o abren otra ventana	Abre el editor con `abrir_editor.bat`: otro Notepad++ instalado en el PC no tiene la tecla configurada. Comprueba que la pestaña activa es el banco de pruebas. Alternativa: `Plugins > NppExec > Simular`.
Windows bloquea los programas del USB («Windows protegió su PC» o un aviso de directiva)	En ese PC el kit no puede funcionar. Prueba en otro PC, copia la carpeta `VHDL` al disco de tu propio ordenador o usa EDA Playground en el navegador.

PROBLEMA	SOLUCIÓN
El antivirus pone en cuarentena `ghdl.exe` o `gtkwave.exe`	En tu ordenador, restaura el fichero desde el antivirus. En un PC del aula, usa otro PC o EDA Playground.
Una señal vale `U` o `X` en GTKWave	`U`: nadie le ha dado valor todavía; casi siempre falta aplicar el reset al principio o dar valor inicial a las entradas en el banco de pruebas. `X`: dos asignaciones distintas a la misma señal, o una operación con una señal que vale `U`.
GTKWave se abre pero la zona de ondas está vacía	No hay vista guardada: añade las señales (sección 4) y guarda la vista como `tb_xxx.gtkw`.
Las ondas no cambian tras corregir el código	Estás mirando una ventana antigua de GTKWave. Usa `Ctrl` + `Shift` + `R` o mira la ventana nueva.
Todas las señales valen `U`	Estás simulando el diseño en vez del banco de pruebas, o el banco de pruebas no asigna valores a las entradas.
Error extraño con rutas muy largas	GHDL no admite rutas de más de 260 caracteres. Usa nombres de carpeta cortos.

GHDL solo simula

La síntesis, la implementación y la programación de la FPGA se siguen haciendo con Vivado. El kit usa VHDL-2008; si Vivado rechaza algo que GHDL acepta, activa VHDL-2008 en las propiedades del fichero en Vivado.

10 Chuleta de atajos

NOTEPAD++	
`F6`	Guardar todo, compilar, simular y abrir GTKWave
`F9`	Comprobar el fichero activo sin simular
`Ctrl` + `F6`	Repetir la última orden de NppExec
`Ctrl` + `S`	Guardar el fichero
`Ctrl` + `G`	Ir a una línea
`Ctrl` + `Q`	Comentar o descomentar líneas

GTKWAVE	
`Shift` + `Alt` + `F`	Ver toda la simulación (Zoom Best Fit)
`Ctrl` + `+` / `Ctrl` + `-`	Acercar / alejar
`Ctrl` + `Shift` + `R`	Recargar las ondas tras volver a simular
`Ctrl` + `S`	Guardar la vista (.gtkw)
Clic en las ondas	Colocar el cursor y leer los valores

Kit: Notepad++ 8.9.8.1 · NppExec 0.8.12.1 · GHDL 6.0.0 · GTKWave 3.3.100 · Python 3.15.2. Este manual está en el kit como `MANUAL_USUARIO.html` y se abre con cualquier navegador, sin conexión. También se puede leer en línea en jesusomarlacruz.github.io/light-vhdl-kit o descargar en PDF.